

Chapter 9

Exception Handling, Lambda Functions & File Handling

1. What Is an Exception?

An exception is an unexpected event that happens during program execution (runtime) that disrupts the normal flow of instructions.

Example Causes

- Dividing a number by zero.
- Accessing an invalid array index.
- Opening a file that doesn't exist.
- Invalid user input.

When such events occur, an exception is thrown. You can catch it to prevent the program from stopping.

2. Types of Exceptions

2.1 Checked Exceptions (Compile-time Exceptions)

These are exceptions checked and enforced by the compiler. You must handle them using try-catch or throws.

- Examples: IOException, SQLException, ClassNotFoundException, FileNotFoundException

```
try {  
    FileInputStream file = new FileInputStream("data.txt");  
} catch (FileNotFoundException e) {  
    System.out.println("File not found!");  
}
```

2.2 Unchecked Exceptions (Runtime Exceptions)

These occur during runtime and are NOT checked by the compiler.

- Examples: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException

```
int x = 10 / 0; // Throws ArithmeticException
```

2.3 Errors

Errors are serious issues that are not meant to be handled by application code. They occur due to system-level problems and usually crash the program.

- Examples: OutOfMemoryError, StackOverflowError, VirtualMachineError

Summary — Checked vs Unchecked vs Error

Type	Checked by Compiler?	Should Be Handled?	Examples
Checked Exception	Yes	Yes	IOException, SQLException
Unchecked Exception	No	Optional	NullPointerException, ArithmeticException
Error	No	No	OutOfMemoryError, StackOverflowError

3. Exception Handling Methods in Java

Method	Purpose
try	Wraps risky code.
catch	Handles the exception.
finally	Always executes.
throw	Manually throws an exception.
throws	Declares an exception for the caller to handle.

3.1 Handling With try-catch

The code that may cause an exception is placed inside a try block, and the exception is handled inside a catch block.

```
try {
    int a = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("You cannot divide by zero!");
}
```

3.2 Multiple Catch Blocks

You can catch different exception types separately.

```
try {
    int[] arr = new int[3];
    System.out.println(arr[5]);
} catch (ArithmeticException e) {
    System.out.println("Math error!");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index is invalid!");
}
```

3.3 Using the finally Block

The finally block always runs — whether an exception occurs or not. It is commonly used for closing resources (files, database connections, etc.).

```
try {
    System.out.println("Inside try block");
}
```

```

} catch (Exception e) {
    System.out.println("Exception caught");
} finally {
    System.out.println("This will always execute");
}

```

finally Block Example — with Division

```

public class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (Exception e) {
            System.out.println("Error occurred");
        } finally {
            System.out.println("Finally block executed");
        }
    }
}

```

Error occurred

Finally block executed

3.4 Using the throw Keyword

The throw keyword is used to explicitly throw an exception from a method or block of code.

- It is used inside a method.
- It throws ONE exception object at a time.
- It transfers control to the nearest matching catch block.

Syntax: throw new ExceptionType("Error message");

```

public class Demo {
    public static void main(String[] args) {
        int age = 16;
        if (age < 18) {
            throw new ArithmeticException("Not Eligible to Vote");
        }
        System.out.println("Eligible");
    }
}

```

Output: Exception in thread "main" java.lang.ArithmeticException: Not Eligible to Vote — here, throw immediately creates and throws an ArithmeticException, ending the program since it's never caught.

3.5 Using the throws Keyword

The throws keyword is used to declare that a method MAY throw one or more exceptions.

- It is written after the method name in the method signature.

- It informs the compiler and the calling method that an exception may occur.
- It is mainly used with checked exceptions.

Syntax: returnType methodName() throws ExceptionType { // code }

```
class Demo {
    static void checkAge(int age) throws Exception {
        if (age < 18) {
            throw new Exception("Not Eligible");
        }
        System.out.println("Eligible");
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output: Not Eligible

Why Is new Used When Throwing an Exception?

Because exceptions are objects in Java, and to throw an exception, we must first create (instantiate) an object of that exception class.

- new creates an object: new ArithmeticException("message") creates an exception object in memory.
- throw throws that object: throw sends the exception object to the JVM.

4. Difference Between throw and throws

Easy way to remember:

- throw = "I am throwing the exception." (an action — actually throws it, right now)
- throws = "This method MAY throw an exception." (a declaration/promise on the method signature)

```
void validate() throws Exception { // Promise that this method may
    throw an exception
    throw new Exception("Error"); // Actually throws the exception
}
```

Example — throw Without throws (Unchecked Exception)

```
class Demo {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Not Eligible");
        }
    }
}
```

```

    }
    System.out.println("Eligible");
}

public static void main(String[] args) {
    checkAge(15);
}
}

```

Output: Exception in thread "main" java.lang.ArithmeticException: Not Eligible (note: unchecked exceptions like ArithmeticException don't require throws in the method signature, unlike checked exceptions).

5. Custom Exception (User-defined Exception)

A Custom Exception is an exception that you create yourself to represent application-specific errors. Java allows this by extending the Exception class (for checked exceptions) or RuntimeException (for unchecked exceptions).

Why Use Custom Exceptions?

- Built-in exceptions do not describe your specific error.
- You want clear, meaningful error messages.
- You want to enforce business rules (e.g., "Age must be > 18").

Example — Custom Exception (Invalid Marks)

```

class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) {
        super(msg);
    }
}

public class Test {
    static void validate(int marks) throws InvalidMarksException {
        if (marks > 100 || marks < 0) {
            throw new InvalidMarksException("Invalid Marks");
        }
        System.out.println("Marks Accepted");
    }

    public static void main(String[] args) {
        try {
            validate(120);
        } catch (InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Output: Invalid Marks

Simple Example — Checked Custom Exception (Age)

Step 1: create the custom exception class.

```
class AgeException extends Exception {  
    public AgeException(String message) {  
        super(message);  
    }  
}
```

Step 2: use it.

```
public class Test {  
    static void checkAge(int age) throws AgeException {  
        if (age < 18) {  
            throw new AgeException("Age must be 18 or above");  
        }  
        System.out.println("Accepted");  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (AgeException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output: Age must be 18 or above

6. Combined Programs (try, catch, throw, throws, finally, Custom Exceptions)

6.1 Age Check System

```
class AgeException extends Exception {  
    public AgeException(String msg) {  
        super(msg);  
    }  
}  
  
public class CombinedExample {  
    static void checkAge(int age) throws AgeException {  
        if (age < 18) {  
            throw new AgeException("Age must be 18 or above!");  
        }  
        System.out.println("Age is valid.");  
    }  
}
```

```
}  
}
```

6.2 Bank Withdrawal System

In this program: if the withdrawal amount is more than the balance, we throw a custom exception. The method declares throws, we handle it in try-catch, and the finally block always runs.

```
// Custom exception for insufficient balance  
class InsufficientBalanceException extends Exception {  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}  
  
public class BankExample {  
    // Method that may throw an exception  
    static void withdraw(int balance, int amount) throws  
    InsufficientBalanceException {  
        if (amount > balance) {  
            throw new InsufficientBalanceException(  
                "Insufficient Balance! Cannot withdraw " + amount);  
        }  
        System.out.println("Withdrawal successful! Remaining balance:  
" + (balance - amount));  
    }  
  
    public static void main(String[] args) {  
        int balance = 5000;  
        int amountToWithdraw = 6000;  
  
        try {  
            withdraw(balance, amountToWithdraw); // risky method  
        } catch (InsufficientBalanceException e) {  
            System.out.println("Exception Caught: " + e.getMessage());  
        } finally {  
            System.out.println("Thank you for using our bank  
service.");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

*Exception Caught: Insufficient Balance! Cannot withdraw 6000
Thank you for using our bank service.
Program continues...*

6.3 Student Registration — Two Custom Exceptions

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}

class InvalidMarksException extends Exception {
    public InvalidMarksException(String msg) {
        super(msg);
    }
}

public class StudentRegistration {
    static void registerStudent(int age, int marks)
        throws InvalidAgeException, InvalidMarksException {
        if (age < 18)
            throw new InvalidAgeException("Student must be at least 18
years old.");
        if (marks < 0 || marks > 100)
            throw new InvalidMarksException("Marks should be between 0
and 100.");
        System.out.println("Student Registered Successfully.");
    }

    public static void main(String[] args) {
        try {
            registerStudent(16, 95);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        } catch (InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output: Student must be at least 18 years old. (age is checked first, so the marks check never runs)

6.4 Student Login — Custom + Built-in Exception Together

```
class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String msg) {
        super(msg);
    }
}
```



```

public class StudentLogin {
    static void login(String username, String password)
        throws InvalidPasswordException {
        if (username == null)
            throw new NullPointerException("Username cannot be
null.");
        if (!password.equals("admin123"))
            throw new InvalidPasswordException("Incorrect Password.");
        System.out.println("Login Successful");
    }

    public static void main(String[] args) {
        try {
            login(null, "abc");
        } catch (NullPointerException e) {
            System.out.println(e);
        } catch (InvalidPasswordException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Output: java.lang.NullPointerException: Username cannot be null.

6.5 Student Registration with User Input (Two Sample Runs)

```

import java.util.Scanner;

class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) { super(msg); }
}
class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) { super(msg); }
}

public class Register {
    static void register(int age, int marks)
        throws InvalidAgeException, InvalidMarksException {
        if (age < 18)
            throw new InvalidAgeException("Age must be 18 or above.");
        if (marks < 0 || marks > 100)
            throw new InvalidMarksException("Marks should be between 0
and 100.");
        System.out.println("Student Registered Successfully.");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```

```

        System.out.print("Enter Student Age: ");
        int age = sc.nextInt();
        System.out.print("Enter Student Marks: ");
        int marks = sc.nextInt();

        try {
            register(age, marks);
        } catch (InvalidAgeException | InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Sample Output 1:

*Enter Student Age: 16
Enter Student Marks: 88
Age must be 18 or above.*

Sample Output 2:

*Enter Student Age: 20
Enter Student Marks: 120
Marks should be between 0 and 100.*

7. Sample Exercises (Detailed)

Exercise A — Student Result System

Requirements: marks must be between 0 and 100; total marks must not exceed 300; if the student fails in any subject (<35), throw `FailException`; if invalid marks entered (negative or >100), throw `InvalidMarksException`.

Possible exceptions: `InvalidMarksException` (marks <0 or >100), `FailException` (any subject <35), `NullPointerException` (if no marks given), `ArithmeticException` (division by zero when calculating percentage).

```

import java.util.Scanner;

// Custom Exception for Invalid Marks
class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) { super(msg); }
}

// Custom Exception for Failed Student
class FailException extends Exception {
    FailException(String msg) { super(msg); }
}

public class StudentResult {
    static void calculateResult(Integer m1, Integer m2, Integer m3)

```

```

        throws InvalidMarksException, FailException {

// NullPointerException
if (m1 == null || m2 == null || m3 == null) {
    throw new NullPointerException("Marks cannot be null.");
}

// InvalidMarksException
if (m1 < 0 || m1 > 100 || m2 < 0 || m2 > 100 || m3 < 0 || m3 >
100) {
    throw new InvalidMarksException("Marks should be between 0
and 100.");
}

// FailException
if (m1 < 35 || m2 < 35 || m3 < 35) {
    throw new FailException("Student Failed (Marks below
35).");
}

int total = m1 + m2 + m3;
if (total > 300) {
    throw new InvalidMarksException("Total marks cannot exceed
300.");
}

int subjects = 3;
double percentage = total / subjects; // note: integer
division here

System.out.println("\n----- RESULT -----");
System.out.println("Total Marks = " + total);
System.out.println("Percentage = " + percentage + "%");
System.out.println("Result = PASS");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    try {
        System.out.print("Enter Marks of Subject 1: ");
        int m1 = sc.nextInt();
        System.out.print("Enter Marks of Subject 2: ");
        int m2 = sc.nextInt();
        System.out.print("Enter Marks of Subject 3: ");
        int m3 = sc.nextInt();
    }
}

```

```

        calculateResult(m1, m2, m3);

    } catch (InvalidMarksException e) {
        System.out.println("InvalidMarksException: " +
e.getMessage());
    } catch (FailException e) {
        System.out.println("FailException: " + e.getMessage());
    } catch (ArithmeticException e) {
        System.out.println("ArithmeticException: Division by
zero.");
    } catch (NullPointerException e) {
        System.out.println("NullPointerException: " +
e.getMessage());
    } catch (Exception e) {
        System.out.println("Invalid Input.");
    }
    sc.close();
}
}

```

Sample Output 1 (Pass):

Enter Marks of Subject 1: 90

Enter Marks of Subject 2: 80

Enter Marks of Subject 3: 85

----- RESULT -----

Total Marks = 255

Percentage = 85.0%

Result = PASS

Sample Output 2 (FailException):

Enter Marks of Subject 1: 98

Enter Marks of Subject 2: 20

Enter Marks of Subject 3: 80

FailException: Student Failed (Marks below 35).

Sample Output 3 (InvalidMarksException):

Enter Marks of Subject 1: 120

Enter Marks of Subject 2: 80

Enter Marks of Subject 3: 78

InvalidMarksException: Marks should be between 0 and 100.

Sample Output 4 (Invalid Input):

Enter Marks of Subject 1: abc

Invalid Input.

Exercise B — Login Authentication System

A simple login system: username must match a stored username, password must be at least 8 characters. If username is wrong, throw `InvalidUserException`. If password is wrong, throw `InvalidPasswordException`. If password is too short, throw `WeakPasswordException`.

Possible exceptions: `InvalidUserException` (wrong username), `InvalidPasswordException` (incorrect password), `WeakPasswordException` (password length < 8), `NullPointerException` (if user enters nothing).

Exercise C — ATM Simulation

ATM features: check balance, withdraw money, minimum balance of 500 required. If withdrawal amount > balance, throw `InsufficientBalanceException`. If withdrawal < 100 or not a multiple of 100, throw `InvalidAmountException`. If PIN is wrong, throw `InvalidPinException`.

Possible exceptions: `InvalidPinException` (wrong PIN), `InsufficientBalanceException` (not enough money), `InvalidAmountException` (amount not valid), `ArithmeticException` (if withdrawal causes a negative balance).

Exercise D — Railway Ticket Reservation

Problem: write a Java program for railway ticket booking. Requirements: accept passenger name, age, train number, and number of tickets; age should be at least 5 years; train number should be valid; maximum 6 tickets per booking.

Custom Exceptions: `InvalidTrainException`, `InvalidAgeException`, `TicketLimitException`.

Predefined Exceptions: `InputMismatchException`, `NullPointerException`.

8. Lambda Functions (Introduction)

In Java, a lambda function (or lambda expression) is a short way to write anonymous methods (methods without a name). Lambdas were introduced in Java 8 to make code shorter, cleaner, and easier — especially when working with functional interfaces, collections, or streams.

Syntax: (parameter) -> { body }

Simple Explanation — Old Java vs Lambda

Without lambda (old Java) — using an anonymous inner class:

```
interface Say {
    void speak();
}

Say s = new Say() {
    public void speak() {
        System.out.println("Hello!");
    }
};
```

With lambda:

```
Say s = () -> System.out.println("Hello!");
```

Lambda advantages: shorter, cleaner, easier.

Example 1 — Lambda with No Parameters

```
interface Greeting {
    void sayHello();
}

public class LambdaExample1 {
    public static void main(String[] args) {
        // Lambda Expression
        Greeting g = () -> System.out.println("Hello from Lambda!");
        g.sayHello();
    }
}
```

Output: Hello from Lambda!

Example 2 — Lambda with Two Parameters

```
interface Add {
    int add(int a, int b);
}

public class LambdaExample3 {
    public static void main(String[] args) {
        Add obj = (a, b) -> a + b;
        System.out.println("Sum = " + obj.add(10, 20));
    }
}
```

Output: Sum = 30

9. File Handling

File handling in Java allows programs to create, read, write, append, and delete files stored on disk. Java provides classes for handling files mainly inside the `java.io` package:

- `File`
- `FileReader`, `BufferedReader`
- `FileWriter`, `BufferedWriter`
- `FileInputStream`, `FileOutputStream`

9.1 Simple Program to Create & Write to a File (`FileWriter`)

```
import java.io.FileWriter;
import java.io.IOException;

public class SimpleFileWrite {
    public static void main(String[] args) {
        try {
```

```

        // Create FileWriter object to open (or create) a file
        FileWriter file = new FileWriter("myfile.txt");

        // Write content into the file
        file.write("Hello, this is a simple file writing
example.");

        // Close file after writing
        file.close();

        System.out.println("File written successfully!");
    } catch (IOException e) {
        System.out.println("An error occurred.");
    }
}
}

```

How It Works

1. FileWriter opens the file myfile.txt (if the file does not exist, it will be created).
2. write() stores the text inside the file.
3. close() saves and closes the file.
4. A confirmation message is printed after completion.

Note: by default, FileWriter("myfile.txt") OVERWRITES the file's previous content each time the program runs, since no append flag is given.

9.2 Why Use BufferedWriter?

BufferedWriter writes text to a file faster by collecting data in a buffer before writing it all at once, instead of writing character-by-character.

Without BufferedWriter	With BufferedWriter
Writes directly to file each time	Stores data in memory first
Slower for large content	Faster and more efficient
More disk operations	Fewer disk operations

Key Features

- Speeds up writing large text files.
- Supports the newLine() method to easily add new lines.
- Works together with FileWriter, FileOutputStream, etc.

In short: BufferedWriter = Faster writing + Less disk access.

```

BufferedWriter bw = new BufferedWriter(new FileWriter("sample.txt"));
bw.write("Hello from BufferedWriter!");
bw.close();

```

Example — BufferedWriter with Append Mode

Passing true as the second argument to FileWriter enables APPEND mode — new writes are added to the end of the file instead of overwriting it.

```
import java.io.BufferedWriter;
import java.io.FileWriter;

public class MainClass {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("output.txt", true); //
            true = append mode
            BufferedWriter bw = new BufferedWriter(fw);

            bw.write("Hey Man");
            bw.newLine();
            bw.write("Okay");
            bw.close();

            System.out.println("Success");
        } catch (Exception e) {
            System.out.println("Something has happened");
        }
    }
}
```

9.3 FileReader — Reading a File Character by Character

Basic version — reads a single character and prints its ASCII value:

```
import java.io.FileNotFoundException;
import java.io.FileReader;

public class MainClass {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("output.txt");
            int c = fr.read();
            System.out.println(c);
            fr.close();
        } catch (Exception e) {
        }
    }
}
```

What this code does: opens output.txt using FileReader; reads ONE character from the file (fr.read() returns its ASCII/Unicode value as an int, not the character itself); prints that integer to the console. For example, if the first character in the file is 'H', it prints 72 (its ASCII code).

Printing the Actual Characters Instead of ASCII Values

Casting the int result to (char) converts the ASCII code back into the readable character:

```
import java.io.FileNotFoundException;
import java.io.FileReader;

public class MainClass {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("output.txt");
            int c = fr.read();
            System.out.print((char) c);

            c = fr.read();
            System.out.print((char) c);

            c = fr.read();
            System.out.print((char) c);

            c = fr.read();
            System.out.print((char) c);

            c = fr.read();
            System.out.print((char) c);

            c = fr.read();
            System.out.print(c); // no cast here - prints the ASCII
number instead of the character

            fr.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Note: the last read() call intentionally omits the (char) cast, printing the raw ASCII number instead of a character — a reminder that read() always returns an int, and casting to char is a deliberate step you must remember every time.

This approach of calling fr.read() manually, one character at a time, works but is clearly repetitive and impractical for anything longer than a few characters — which is exactly the problem BufferedReader solves next.

9.4 BufferedReader — Reading a File Efficiently

BufferedReader is a class used to read text from a file (or input source) efficiently by using a buffer (temporary memory area). It reads large chunks of data at once and stores them in a buffer, instead of reading one character at a time.

Without BufferedReader	With BufferedReader
Reads character-by-character	Reads big chunks of text at once
Slower for large files	Faster and more efficient
More disk operations	Fewer disk operations

Early Version — Reading a Fixed Number of Lines

```
FileReader fr = new FileReader("output.txt");
BufferedReader br = new BufferedReader(fr);
```

```
String line = br.readLine();
System.out.println(line);
```

```
line = br.readLine();
System.out.println(line);
```

```
line = br.readLine();
System.out.println(line);
```

```
fr.close();
```

This works, but only reads a fixed number of lines — if the file has more or fewer lines, this approach breaks down.

Final Version — Reading ALL Lines with a while Loop

```
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.BufferedReader;

public class MainClass {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("output.txt");
            BufferedReader br = new BufferedReader(fr);

            String line = br.readLine();

            while (line != null) {
                System.out.println(line);
                line = br.readLine();
            }

            br.close();
        }
    }
}
```

```

        } catch (Exception e) {
        }
    }
}

```

Logic: `readLine()` returns null once there are no more lines left to read. So the loop keeps printing lines and reading the next one, and automatically stops the moment `readLine()` returns null — this correctly handles a file of ANY length, unlike the fixed-line version above.

10. Tips for This Chapter

1. The fastest way to decide checked vs. unchecked: if the compiler forces you to handle it (with try-catch or throws) before your code will even compile, it's checked. If your code compiles fine either way and the exception only shows up when you actually run it, it's unchecked.
2. `throw` vs `throws` is a grammar distinction as much as a code one: `throw` is a verb (an action that happens inside a method body, right now). `throws` is more like an adjective/warning label (written in the method signature, promising this method MIGHT throw something, without actually throwing anything itself).
3. A method that declares `throws` doesn't have to catch anything itself — it can pass the responsibility of handling the exception up to whatever code called it. That calling code then needs its own try-catch (or its own `throws`, passing it up even further).
4. Order your multiple catch blocks from most specific to most general. A general catch (`Exception e`) placed FIRST would catch everything and make any catch blocks written after it for specific exceptions completely unreachable.
5. The finally block runs no matter what — even if the try block returns a value, or the catch block itself throws a new exception. That's exactly why it's the standard place to close files, database connections, or any other resource that must never be left open.
6. A custom exception class is almost always just a constructor that calls `super(msg)` — the parent `Exception` class already handles storing and returning the message via `getMessage()`. You rarely need to add anything beyond that constructor.
7. When multiple custom exceptions are possible in one method, list every exception type after `throws` (e.g., `throws InvalidAgeException, InvalidMarksException`) — or, in the catch blocks, you can combine them into one block using the pipe symbol: `catch (InvalidAgeException | InvalidMarksException e)`.
8. For `FileReader`, always remember `read()` returns an `int` (the ASCII/Unicode code), not a character — you must cast it with `(char)` to see the actual letter. Forgetting the cast is a common mistake, and deliberately seeing what happens without it (as shown in the chapter's own example) is a good way to internalize this.
9. `BufferedReader`'s `readLine()` returns null exactly once the file has no more lines — this null is precisely what a `while (line != null)` loop checks for, letting you read a file of any length without knowing its line count in advance.
10. Remember the `FileWriter` append flag: `new FileWriter("file.txt")` OVERWRITES existing content, while `new FileWriter("file.txt", true)` APPENDS to the end instead. Forgetting the `true` is a common cause of accidentally erased file content.